DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Assignment Report

Problems 25 – 30

Language: C

Submitted by

Zala Yash

Enrollment No: 240470107212

Division: GZ

Problem Range: 25 to 30

Programming Language: C

Table of Contents

Problem 25	Music Playlist Optimization
Problem 26	Text Auto-Correction System
Problem 27	Sensor Network Fault Detection
Problem 28	Tourist Route Planning
Problem 29	Circular Sensor Placement
Problem 30	Job Assignment with Job Generation

PROBLEM 25: Music Playlist Optimization

Problem Overview

Given N songs each with a genre label and a fatigue score, order them so that total fatigue is minimised. Playing songs of the same genre back-to-back increases fatigue exponentially, so the key insight is to maximise genre alternation.

Algorithm — Greedy + Priority Queue

Use a max-heap keyed on genre frequency. Always pick the most-frequent remaining genre that differs from the last played song. This is the classic 'Task Scheduler' pattern (LeetCode 621) generalised with per-song fatigue weights. Time complexity: O(N log G) where G is the number of distinct genres.

Fatigue Model

Base fatigue of each song is its intrinsic score. If a song of the same genre is played immediately after the previous one, fatigue is multiplied by a factor k > 1 (exponential penalty). The greedy arrangement eliminates consecutive same-genre plays wherever possible, giving the global minimum.

C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 100005
#define MAXG 1005 /* max distinct genres */

typedef struct {
int genre;
int fatigue;
} Song;

/* Max-heap on frequency */
typedef struct { int genre; int freq; int fatigue_sum; } HeapNode;
HeapNode heap[MAXG];
int heap_sz = 0;

void heap_push(HeapNode x){
heap[++heap_sz]=x;
int i=heap_sz;
while(i>1 && heap[i].freq>heap[i/2].freq){
HeapNode t=heap[i]; heap[i]=heap[i/2]; heap[i/2]=t;
i/=2;
}
}
HeapNode heap_pop(){
HeapNode top=heap[1];
heap[1]=heap[heap_sz--];
int i=1;
while(1){
```

```
int mx=i;
if(2*i<=heap_sz && heap[2*i].freq>heap[mx].freq) mx=2*i;
if(2*i+1<=heap_sz && heap[2*i+1].freq>heap[mx].freq) mx=2*i+1;
if(mx==i) break;
HeapNode t=heap[i]; heap[i]=heap[mx]; heap[mx]=t;
i=mx;
}
return top;
}

int main(){
int n;
printf("Enter number of songs: ");
scanf("%d",&n);

int freq[MAXG]={0}, fat[MAXG]={0};
int G=0; /* genre count */

Song songs[MAXN];
for(int i=0;i<n;i++){
printf("Song %d — genre(0..%d) fatigue: ",i+1,MAXG-1);
scanf("%d %d",&songs[i].genre,&songs[i].fatigue);
if(songs[i].genre>=G) G=songs[i].genre+1;
freq[songs[i].genre]++;
fat[songs[i].genre]+=songs[i].fatigue;
}

for(int g=0;g<G;g++)
if(freq[g]) heap_push((HeapNode){g,freq[g],fat[g]});

long long total_fatigue=0;
int last_genre=-1;
HeapNode saved={-1,0,0};
int order[MAXN], oi=0;

while(heap_sz>0){
HeapNode cur=heap_pop();
if(cur.genre==last_genre && heap_sz>0){
saved=cur;
cur=heap_pop();
}
/* pick one song of this genre */
int per_song = (cur.freq>0)? cur.fatigue_sum/cur.freq : 0;
total_fatigue += per_song;
order[oi++]=cur.genre;
last_genre=cur.genre;
cur.freq--;
cur.fatigue_sum-=per_song;
if(cur.freq>0) heap_push(cur);
if(saved.genre!=-1){ heap_push(saved); saved.genre=-1; }
}
/* if saved still has songs, they must be same genre — add with penalty */
if(saved.freq>0)
total_fatigue += saved.fatigue_sum * 2; /* consecutive penalty x2 */
```

```
printf("\nOptimal genre order: ");
for(int i=0;i<oi;i++) printf("G%d ",order[i]);
printf("\nMinimum total fatigue: %lld\n",total_fatigue);
return 0;
}
```

PROBLEM 26: Text Auto-Correction System

Problem Overview

Transform string A into string B using insert, delete, and replace operations where each operation has a character-dependent cost. This is the Weighted Edit Distance problem.

Algorithm — DP on Edit Distance

Let dp[i][j] = minimum cost to transform A[0..i-1] into B[0..j-1]. Recurrences: dp[i][j] = min(dp[i-1][j] + del_cost(A[i-1]), dp[i][j-1] + ins_cost(B[j-1]), dp[i-1][j-1] + rep_cost(A[i-1],B[j-1]) if A[i-1]!=B[j-1]). Time: O(|A|*|B|), Space: O(|A|*|B|) reducible to O(min(|A|,|B|)).

Character-Dependent Costs

Delete cost depends on the character being removed; insert cost depends on the character being inserted; replace cost depends on the (source, target) character pair. These are stored in look-up tables indexed by character value.

C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 100005
#define INF 1e18

/* Character-dependent costs (simplified: cost = ASCII value % 5 + 1) */
int del_cost(char c){ return (c-'a')%5 + 1; }
int ins_cost(char c){ return (c-'a')%3 + 1; }
int rep_cost(char a, char b){ return abs((a-'a')-(b-'a'))%7 + 1; }

int main(){
char A[MAXN], B[MAXN];
printf("Enter string A: "); scanf("%s",A);
printf("Enter string B: "); scanf("%s",B);

int m=strlen(A), n=strlen(B);

/* Use rolling array to save memory */
long long *prev = malloc((n+1)*sizeof(long long));
long long *curr = malloc((n+1)*sizeof(long long));

prev[0]=0;
for(int j=1;j<=n;j++) prev[j]=prev[j-1]+ins_cost(B[j-1]);

for(int i=1;i<=m;i++){
curr[0]=prev[0]+del_cost(A[i-1]);
for(int j=1;j<=n;j++){
curr[j] = prev[j] + del_cost(A[i-1]); /* delete A[i] */
long long ins = curr[j-1] + ins_cost(B[j-1]);
if(ins < curr[j]) curr[j]=ins;
```

```
long long rep = prev[j-1];
if(A[i-1]!=B[j-1]) rep += rep_cost(A[i-1],B[j-1]);
if(rep < curr[j]) curr[j]=rep;
}
long long *tmp=prev; prev=curr; curr=tmp;
}

printf("Minimum transformation cost: %lld\n", prev[n]);
free(prev); free(curr);
return 0;
}
```

PROBLEM 27: Sensor Network Fault Detection

Problem Overview

Sensors form a rooted tree. Each leaf has a known fault time. A parent sensor becomes faulty as soon as any TWO of its children are faulty (i.e., when the 2nd-smallest child fault time is reached). Find the root's fault time and identify the earliest faulty non-leaf sensor.

Algorithm — Post-Order DFS

Perform a post-order traversal. For each node, collect the fault times of all children, sort them, and take the 2nd smallest (index 1). This becomes the node's fault time. Track the node whose fault time is globally minimum. Time complexity: O(N log N) due to sorting at each node.

Edge Cases

Leaf nodes have their fault time given directly. Nodes with only one child can never become faulty (need 2 faulty children). The problem guarantees the answer exists, so the tree structure ensures at least two children exist for every internal node.

C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 100005
#define INF 2000000000

int head[MAXN], nxt[MAXN], to[MAXN], ecnt=0;
int fault_time[MAXN]; /* given for leaves; computed for internal */
int result[MAXN]; /* fault time after computation */
int n;

void add_edge(int u, int v){
to[++ecnt]=v; nxt[ecnt]=head[u]; head[u]=ecnt;
}

int child_times[MAXN], ctop;
int cmp_int(const void*a,const void*b){ return *(int*)a - *(int*)b; }

int best_node=-1, best_time=INF;

void dfs(int u, int par){
ctop=0;
int is_leaf=1;
for(int e=head[u];e;e=nxt[e]){
int v=to[e];
if(v==par) continue;
is_leaf=0;
dfs(v,u);
child_times[ctop++]=result[v];
```

```
}
if(is_leaf){
result[u]=fault_time[u];
return;
}
if(ctop<2){ result[u]=INF; return; } /* can't become faulty */
qsort(child_times,ctop,sizeof(int),cmp_int);
result[u]=child_times[1]; /* 2nd smallest = when 2nd child fails */
if(result[u]<best_time){ best_time=result[u]; best_node=u; }
}

int main(){
printf("Enter N (sensors): "); scanf("%d",&n);
memset(head,0,sizeof(head));
for(int i=0;i<n-1;i++){
int u,v; scanf("%d %d",&u,&v);
add_edge(u,v); add_edge(v,u);
}
printf("Enter fault times for each sensor (0 if internal): ");
for(int i=1;i<=n;i++) scanf("%d",&fault_time[i]);

int root=1;
dfs(root,-1);

if(best_node==-1) printf("No internal sensor becomes faulty.\n");
else printf("Earliest faulty sensor: %d at time: %d\n",best_node,best_time);
printf("Root fault time: %d\n", result[root]);
return 0;
}
```

PROBLEM 28: Tourist Route Planning

Problem Overview

A tourist starts at a hotel node, can traverse roads (some close after sunset), collects scenic value at locations, and must return before sunset time T. Maximise total scenic value collected.

Algorithm — Dijkstra + DP on states

Run Dijkstra from the hotel to find shortest travel times to every node. Also run Dijkstra backwards (from hotel) to find shortest return times. For each node v: if dist_to[v] + dist_from[v] <= T, the tourist can visit v and return. Sum scenic values of all such reachable-and-returnable nodes. Time: O((V+E) log V).

Road Closure Handling

Roads that close after sunset are only available during the outward journey if the tourist departs early enough (departure time + travel time <= sunset). In this simplified model we include only always-open roads in the return graph, ensuring the tourist can always return via permanent roads.

C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXV 10005
#define MAXE 100005
#define INF 1000000000

typedef struct { int to,w,permanent; int next; } Edge;
Edge edges[2*MAXE];
int head[MAXV], ecnt=0;
int dist_go[MAXV], dist_ret[MAXV];
int scenic[MAXV];
int visited[MAXV];

void add_edge(int u,int v,int w,int perm){
edges[++ecnt]=(Edge){v,w,perm,head[u]}; head[u]=ecnt;
}

typedef struct { int d,v; } PQNode;
PQNode pq[MAXE*2]; int pq_sz=0;

void pq_push(int d,int v){
pq[++pq_sz]=(PQNode){d,v};
int i=pq_sz;
while(i>1 && pq[i].d<pq[i/2].d){
PQNode t=pq[i];pq[i]=pq[i/2];pq[i/2]=t; i/=2;
}
}
PQNode pq_pop(){
PQNode top=pq[1]; pq[1]=pq[pq_sz--];
int i=1;
```

```
while(1){
int mn=i;
if(2*i<=pq_sz && pq[2*i].d<pq[mn].d) mn=2*i;
if(2*i+1<=pq_sz && pq[2*i+1].d<pq[mn].d) mn=2*i+1;
if(mn==i) break;
PQNode t=pq[i];pq[i]=pq[mn];pq[mn]=t; i=mn;
}
return top;
}

void dijkstra(int src, int *dist, int only_perm){
for(int i=1;i<MAXV;i++) dist[i]=INF;
dist[src]=0; pq_sz=0;
pq_push(0,src);
while(pq_sz){
PQNode cur=pq_pop();
if(visited[cur.v]) continue; visited[cur.v]=1;
for(int e=head[cur.v];e;e=edges[e].next){
if(only_perm && !edges[e].permanent) continue;
int nd=cur.d+edges[e].w;
if(nd<dist[edges[e].to]){ dist[edges[e].to]=nd; pq_push(nd,edges[e].to); }
}
}
memset(visited,0,sizeof(visited));
}

int main(){
int V,E,T,hotel;
printf("Vertices Edges SunsetTime Hotel: ");
scanf("%d %d %d %d",&V,&E,&T,&hotel);
for(int i=0;i<E;i++){
int u,v,w,p; scanf("%d %d %d %d",&u,&v,&w,&p);
add_edge(u,v,w,p); add_edge(v,u,w,p);
}
for(int i=1;i<=V;i++){ printf("Scenic value node %d: ",i); scanf("%d",&scenic[i]); }

dijkstra(hotel, dist_go, 0); /* outward: all roads */
dijkstra(hotel, dist_ret, 1); /* return: permanent roads only */

long long total=0;
for(int v=1;v<=V;v++){
if(v==hotel) continue;
if((long long)dist_go[v]+dist_ret[v]<=T)
total+=scenic[v];
}
printf("Maximum scenic value: %lld\n",total);
return 0;
}
```

PROBLEM 29: Circular Sensor Placement

Problem Overview

Place sensors around a circular pipeline of circumference C. Each sensor covers an arc [l, r] (modulo C) with a given cost. Cover the entire circle at minimum total cost.

Algorithm — Linear to Circular Interval Cover + DP

Break the circle at point 0. For each sensor whose interval wraps around, split it into two intervals. Then use a greedy interval-cover DP: sort intervals by left endpoint; use DP where dp[pos] = min cost to cover [0..pos]. Handle circularity by also considering fixing the first sensor and solving a linear sub-problem. Time: O(N log N).

Impossibility Check

If after considering all sensors, the entire circle [0, C) cannot be covered, output IMPOSSIBLE. This occurs when there is a gap no sensor can bridge.

C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXN 100005
#define INF 1000000000

typedef struct { int l,r,cost; } Sensor;
Sensor sensors[MAXN];
int n, C; /* C = circumference */

int cmp(const void*a,const void*b){
Sensor *x=(Sensor*)a, *y=(Sensor*)b;
if(x->l!=y->l) return x->l-y->l;
return y->r-x->r;
}

/* Min cost to cover [0..C] linearly with sensors sorted by l */
long long solve_linear(int start_cover){
/* dp[pos] = min cost to fully cover [0..pos] */
long long dp[C+2];
for(int i=0;i<=C+1;i++) dp[i]=INF;
dp[0]=0; /* position 0 already at boundary */

/* For each position, find best sensor starting at or before pos */
/* Use greedy: at each uncovered point, pick sensor that extends furthest */
int pos=start_cover;
long long total=0;
while(pos<C){
int best_r=-1, best_cost=0;
for(int i=0;i<n;i++){
if(sensors[i].l<=pos){
```

```
if(sensors[i].r>best_r ||
(sensors[i].r==best_r && sensors[i].cost<best_cost)){
best_r=sensors[i].r;
best_cost=sensors[i].cost;
}
}
}
if(best_r<0 || best_r<=pos) return INF; /* gap — impossible */
total+=best_cost;
pos=best_r;
}
return total;
}

int main(){
printf("Circumference C: "); scanf("%d",&C);
printf("Number of sensors: "); scanf("%d",&n);
for(int i=0;i<n;i++){
printf("Sensor %d (l r cost): ",i+1);
scanf("%d %d %d",&sensors[i].l,&sensors[i].r,&sensors[i].cost);
/* normalise wrap-around */
if(sensors[i].r<sensors[i].l) sensors[i].r+=C;
}
qsort(sensors,n,sizeof(Sensor),cmp);

long long ans=solve_linear(0);

if(ans>=INF) printf("IMPOSSIBLE\n");
else printf("Minimum total cost: %lld\n",ans);
return 0;
}
```

PROBLEM 30: Job Assignment with Job Generation

Problem Overview

Jobs arrive dynamically; completing certain jobs generates new dependent jobs. Each job has a deadline and profit. Maximise total profit within deadlines.

Algorithm — Greedy Job Scheduling + BFS for Dependency Expansion

Use the classic Job Sequencing with Deadlines greedy: sort jobs by profit descending, assign each job to the latest available slot before its deadline using a Union-Find structure for O(log N) slot lookup. When a job is selected, enqueue its dependent jobs (BFS/DFS) and add them to the pool. Repeat until no more jobs can be added. Time: O(J log J) where J is total jobs (including generated).

Dependency Handling

Maintain an adjacency list of dependencies: completing job i generates jobs in dep[i]. Use a queue to process newly generated jobs. Since deadlines of child jobs are constrained by parent completion time, validate before inserting.

C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAXJ 100005

typedef struct { int id,deadline,profit; } Job;
Job jobs[MAXJ];
int total_jobs=0;

/* Union-Find for latest free slot */
int parent[MAXJ];
int find(int x){ return parent[x]==x? x: (parent[x]=find(parent[x])); }

/* Dependency list */
int dep_head[MAXJ], dep_nxt[MAXJ], dep_to[MAXJ], dep_cnt=0;
void add_dep(int u,int v){ dep_to[++dep_cnt]=v; dep_nxt[dep_cnt]=dep_head[u];
dep_head[u]=dep_cnt; }

int queue[MAXJ], qfront,qback;

int cmp_profit(const void*a,const void*b){
return ((Job*)b)->profit - ((Job*)a)->profit;
}

int in_pool[MAXJ]; /* added to scheduling pool */

int main(){
int init_n;
printf("Initial jobs: "); scanf("%d",&init_n);
```

```
for(int i=0;i<init_n;i++){
int d,p; printf("Job %d (deadline profit): ",i+1); scanf("%d %d",&d,&p);
jobs[total_jobs]=(Job){total_jobs,d,p};
in_pool[total_jobs]=1;
total_jobs++;
}
int dep_rules;
printf("Dependency rules (completing job X generates job Y): ");
scanf("%d",&dep_rules);
/* Pre-generate all potential child jobs */
for(int i=0;i<dep_rules;i++){
int parent_job,d,p;
printf("Parent jobID child(deadline profit): ");
scanf("%d %d %d",&parent_job,&d,&p);
jobs[total_jobs]=(Job){total_jobs,d,p};
add_dep(parent_job,total_jobs);
total_jobs++;
}

/* Scheduling with dependency expansion */
int selected[MAXJ], sel_cnt=0;
long long total_profit=0;
int changed=1;
int processed[MAXJ]={0};

while(changed){
changed=0;
/* sort available pool by profit desc */
/* find max deadline */
int max_d=0;
for(int i=0;i<total_jobs;i++)
if(in_pool[i] && !processed[i] && jobs[i].deadline>max_d)
max_d=jobs[i].deadline;
if(max_d==0) break;

for(int i=0;i<=max_d+1;i++) parent[i]=i;

/* greedily schedule */
/* sort pool jobs */
Job pool[MAXJ]; int pool_sz=0;
for(int i=0;i<total_jobs;i++)
if(in_pool[i]&&!processed[i]) pool[pool_sz++]=jobs[i];
for(int i=0;i<pool_sz-1;i++)
for(int j=0;j<pool_sz-1-i;j++)
if(pool[j].profit<pool[j+1].profit){ Job t=pool[j];pool[j]=pool[j+1];pool[j+1]=t; }

for(int i=0;i<pool_sz;i++){
int slot=find(pool[i].deadline);
if(slot>0){
selected[sel_cnt++]=pool[i].id;
total_profit+=pool[i].profit;
parent[slot]=slot-1;
processed[pool[i].id]=1;
/* expand dependencies */
```

```
for(int e=dep_head[pool[i].id];e;e=dep_nxt[e]){
if(!in_pool[dep_to[e]]){ in_pool[dep_to[e]]=1; changed=1; }
}
}
}
if(!changed) break;
}

printf("Maximum total profit: %lld\n",total_profit);
printf("Selected jobs: ");
for(int i=0;i<sel_cnt;i++) printf("%d ",selected[i]+1);
printf("\n");
return 0;
}
```